

Student 28 **PRINCE RAJ N** 192312284RD

4 / 4 submitted

Q1. Memory saving**CODE**

```
#include <stdio.h>
#include <string.h>

union SensorReading {
    int heartRate;
    int stepCount;
    char sleepQuality[20];
};

struct SensorReadingStruct {
    int heartRate;
    int stepCount;
    char sleepQuality[20];
};

int main() {
    union SensorReading reading;
    struct SensorReadingStruct readingStruct;

    reading.heartRate = 72;
    printf("Transmitted Reading (Heart Rate): %d bpm\n", reading.heartRate);

    reading.stepCount = 4500;
    printf("Transmitted Reading (Step Count): %d steps\n", reading.stepCount);

    strcpy(reading.sleepQuality, "Good");
    printf("Transmitted Reading (Sleep Quality): %s\n", reading.sleepQuality);

    printf("\nSize of union (used for transmission): %lu bytes\n", sizeof(union SensorReading));
    printf("Size of structure (if used instead): %lu bytes\n", sizeof(struct SensorReadingStruct));

    printf("\nMemory Saved by using Union: %lu bytes\n",
           sizeof(struct SensorReadingStruct) - sizeof(union SensorReading));

    return 0;
}
```

OUTPUT

```
Transmitted Reading (Heart Rate): 72 bpm
Transmitted Reading (Step Count): 4500 steps
Transmitted Reading (Sleep Quality): Good

Size of union (used for transmission): 20 bytes
Size of structure (if used instead): 28 bytes

Memory Saved by using Union: 8 bytes
```

Q2. Structure pointer**CODE**

```
#include <stdio.h>
#include <string.h>

union ClaimDetails {
    struct {
        char hospitalName[50];
        float billAmount;
    } health;

    struct {
        char vehicleNumber[20];
        char accidentLocation[50];
    } vehicle;

    struct {
        char propertyAddress[50];
        float damageCost;
    } property;
};

enum ClaimType { HEALTH, VEHICLE, PROPERTY };

struct Claim {
    int claimID;
    char policyHolderName[50];
    char claimDate[15];
    char status[20];
    enum ClaimType type;
    union ClaimDetails details;
};

void updateClaimStatus(struct Claim *c, const char *newStatus) {
    strcpy(c->status, newStatus);
    printf("Claim ID %d status updated to: %s\n", c->claimID, c->status);
}

void displayClaim(struct Claim *c) {
    printf("\n--- Claim Details ---\n");
    printf("Claim ID: %d\n", c->claimID);
    printf("Policy Holder: %s\n", c->policyHolderName);
    printf("Claim Date: %s\n", c->claimDate);
    printf("Status: %s\n", c->status);

    switch (c->type) {
```

```
    case HEALTH:
        printf("Type: Health\n");
        printf("Hospital: %s\n", c->details.health.hospitalName);
        printf("Bill Amount: %.2f\n", c->details.health.billAmount);
        break;
    case VEHICLE:
        printf("Type: Vehicle\n");
        printf("Vehicle Number: %s\n", c->details.vehicle.vehicleNumber);
        printf("Accident Location: %s\n", c->details.vehicle.accidentLocation);
        break;
    case PROPERTY:
        printf("Type: Property\n");
        printf("Property Address: %s\n", c->details.property.propertyAddress);
        printf("Damage Cost: %.2f\n", c->details.property.damageCost);
        break;
}
}

int main() {
    struct Claim c1;

    c1.claimID = 101;
    strcpy(c1.policyHolderName, "Prince Raj");
    strcpy(c1.claimDate, "2025-01-15");
    strcpy(c1.status, "Pending");
    c1.type = HEALTH;
    strcpy(c1.details.health.hospitalName, "City Hospital");
    c1.details.health.billAmount = 25000.50;

    struct Claim *ptr = &c1;

    displayClaim(ptr);

    updateClaimStatus(ptr, "Under Review");
    updateClaimStatus(ptr, "Approved");

    displayClaim(ptr);

    return 0;
}
```

OUTPUT

```
--- Claim Details ---
Claim ID: 101
Policy Holder: Prince Raj
Claim Date: 2025-01-15
Status: Pending
Type: Health
Hospital: City Hospital
Bill Amount: 25000.50
```

SIMATS Engineering • Department of Computer Science and Engineering • CSA0209 — C Programming • 29-08-2026 • Category: CO4_AT2_Case study • Student Submissions Report

Claim ID 101 status updated to: Under Review
Claim ID 101 status updated to: Approved

--- Claim Details ---
Claim ID: 101
Policy Holder: Prince Raj
Claim Date: 2025-01-15
Status: Approved
Type: Health
Hospital: City Hospital
Bill Amount: 25000.50

Q3. Food delivery**CODE**

```
#include <stdio.h>
#include <string.h>

struct CustomerDetails {
    int customerId;
    char name[50];
    char phone[15];
};

struct DeliveryAddress {
    char street[50];
    char city[30];
    char pincode[10];
};

struct RestaurantDetails {
    int restaurantId;
    char name[50];
};

union PaymentDetails {
    struct {
        char walletId[20];
        float balanceUsed;
    } wallet;

    struct {
        char cardNumber[20];
        char cardHolder[30];
    } card;

    struct {
        char codConfirmedBy[30];
    } cod;
};

enum PaymentType { WALLETT, CARD, COD };

struct Order {
    int orderId;
    struct CustomerDetails customer;
    struct RestaurantDetails restaurant;
    struct DeliveryAddress address;
    enum PaymentType paymentType;
};
```

```
union PaymentDetails payment;
};

int main() {
    struct Order order1;

    order1.orderId = 101;

    strcpy(order1.customer.name, "Prince Raj");
    strcpy(order1.customer.phone, "9876543210");
    order1.customer.customerId = 1;

    strcpy(order1.restaurant.name, "Spice Corner");
    order1.restaurant.restaurantId = 55;

    strcpy(order1.address.street, "12 MG Road");
    strcpy(order1.address.city, "Chennai");
    strcpy(order1.address.pincode, "600001");

    order1.paymentType = WALLET;
    strcpy(order1.payment.wallet.walletId, "WAL123");
    order1.payment.wallet.balanceUsed = 250.50;

    printf("Order ID: %d\n", order1.orderId);
    printf("Customer: %s (%s)\n", order1.customer.name, order1.customer.phone);
    printf("Restaurant: %s\n", order1.restaurant.name);
    printf("Delivery Address: %s, %s - %s\n",
           order1.address.street, order1.address.city, order1.address.pincode);

    if (order1.paymentType == WALLET) {
        printf("Payment via Wallet: ID=%s, Amount=%.2f\n",
               order1.payment.wallet.walletId, order1.payment.wallet.balanceUsed);
    } else if (order1.paymentType == CARD) {
        printf("Payment via Card: %s (%s)\n",
               order1.payment.card.cardNumber, order1.payment.card.cardHolder);
    } else {
        printf("Payment via COD, confirmed by: %s\n",
               order1.payment.cod.codConfirmedBy);
    }

    return 0;
}
```

OUTPUT

Order ID: 101
Customer: Prince Raj (9876543210)
Restaurant: Spice Corner
Delivery Address: 12 MG Road, Chennai - 600001
Payment via Wallet: ID=WAL123, Amount=250.50

Q4. Digital wallet application**CODE**

```
#include <stdio.h>
#include <string.h>

union CurrencyAmount {
    float standardAmount;
    double cryptoAmount;
    long rewardPoints;
};

enum CurrencyType {
    STANDARD,
    CRYPTO,
    REWARD_POINTS
};

struct Transaction {
    int transactionID;
    char timestamp[20];
    int userID;
    enum CurrencyType type;
    union CurrencyAmount amount;
};

void displayTransaction(struct Transaction t) {
    printf("Transaction ID: %d\n", t.transactionID);
    printf("Timestamp: %s\n", t.timestamp);
    printf("User ID: %d\n", t.userID);

    switch (t.type) {
        case STANDARD:
            printf("Currency Type: Standard\n");
            printf("Amount: %.2f\n", t.amount.standardAmount);
            break;
        case CRYPTO:
            printf("Currency Type: Cryptocurrency\n");
            printf("Amount: %.8lf\n", t.amount.cryptoAmount);
            break;
        case REWARD_POINTS:
            printf("Currency Type: Reward Points\n");
            printf("Amount: %ld\n", t.amount.rewardPoints);
            break;
    }
    printf("Size of Transaction struct: %lu bytes\n\n", sizeof(struct Transaction));
}
```

```
int main() {
    struct Transaction t1;
    t1.transactionID = 101;
    strcpy(t1.timestamp, "2024-06-01 10:00");
    t1.userID = 5001;
    t1.type = STANDARD;
    t1.amount.standardAmount = 1500.75;

    struct Transaction t2;
    t2.transactionID = 102;
    strcpy(t2.timestamp, "2024-06-01 10:05");
    t2.userID = 5002;
    t2.type = CRYPTO;
    t2.amount.cryptoAmount = 0.00452310;

    struct Transaction t3;
    t3.transactionID = 103;
    strcpy(t3.timestamp, "2024-06-01 10:10");
    t3.userID = 5003;
    t3.type = REWARD_POINTS;
    t3.amount.rewardPoints = 25000;

    displayTransaction(t1);
    displayTransaction(t2);
    displayTransaction(t3);

    return 0;
}
```

OUTPUT

```
Transaction ID: 101
Timestamp: 2024-06-01 10:00
User ID: 5001
Currency Type: Standard
Amount: 1500.75
Size of Transaction struct: 40 bytes

Transaction ID: 102
Timestamp: 2024-06-01 10:05
User ID: 5002
Currency Type: Cryptocurrency
Amount: 0.00452310
Size of Transaction struct: 40 bytes

Transaction ID: 103
Timestamp: 2024-06-01 10:10
User ID: 5003
Currency Type: Reward Points
Amount: 25000
Size of Transaction struct: 40 bytes
```